카담(CADAM) 프로젝트 — Cursor AI 바이브코딩 완전 초보 가이드

> **대상:** 코딩 경험 없는 비개발자 **목표:** Cursor AI 설치부터 카담 프로젝트 첫 화면이 뜰 때까지 **작성일:** 2026-03-11

🔵 Phase 0: 사전 준비물 설치 (Cursor 열기 전에 해야 할 것들)

Cursor AI는 "코드 편집기"입니다. 하지만 코드 편집기만으로는 웹사이트가 만들어지지 않습니다.
요리에 비유하면 Cursor는 "주방(도구)"이고, 아래 프로그램들은 "가스레인지, 냉장고, 수도(인프라)"입니다.

0-1. Node.js 설치

이것은 무엇인가? JavaScript를 컴퓨터에서 실행할 수 있게 해주는 프로그램입니다. Next.js(우리 프로젝트의 핵심 프레임워크)가 Node.js 위에서 돌아갑니다.

설치 방법:

1. https://nodejs.org 접속
2. **LTS** 버전 다운로드 (숫자가 짝수인 것, 예: 22.x.x)
3. 다운받은 파일 실행 → "Next" 계속 클릭 → 설치 완료

설치 확인 (터미널에서):

- Mac: `Cmd + Space` → "터미널" 검색 → 열기
- Windows: `Win + R` → `cmd` 입력 → 열기

```bash
node --version    # v22.x.x 같은 숫자가 나오면 성공
npm --version     # 10.x.x 같은 숫자가 나오면 성공
```

> **[왜 필요한가 — 사실]** PRD 기술 스택에 "Next.js 15"가 명시되어 있고, Next.js는 Node.js 런타임 위에서만 작동합니다.

0-2. Git 설치

이것은 무엇인가? 코드의 "버전 관리" 도구입니다. 문서 작업할 때 "최종.docx", "진짜최종.docx" 대신 변경 이력을 체계적으로 관리해줍니다.

설치 방법:

- Mac: 터미널에 `git --version` 입력하면 자동 설치 안내가 뜸
- Windows: https://git-scm.com 에서 다운로드 → 설치 (모든 옵션 기본값 OK)

설치 확인:

```bash
git --version     # git version 2.x.x 나오면 성공
```

0-3. Cursor AI 설치

이것은 무엇인가? VS Code(마이크로소프트의 코드 편집기)를 기반으로 만든 AI 코딩 전용 에디터입니다. 채팅으로 AI에게 "이런 코드 만들어줘"라고 지시하면 코드를 생성/수정해줍니다.

설치 방법:

1. https://www.cursor.com 접속
2. "Download" 버튼 클릭 (Mac/Windows 자동 감지)
3. 다운받은 파일 실행 → 설치
4. 처음 실행 시 계정 생성 (구글 로그인 가능)

요금:

- Free: 월 2,000회 코드 자동완성 + 50회 느린 AI 요청
- Pro: 월 $20 — 무제한 자동완성 + 500회 빠른 AI 요청 (추천)

> **[추론]** 카담 프로젝트는 파일 50개 이상, 수백 번의 AI 요청이 필요하므로 Pro 플랜을 사용하는 것이 현실적입니다.

🔵 Phase 1: 프로젝트 생성 (Next.js 초기화)

1-1. 프로젝트 폴더를 만들 위치 정하기

사고 과정:

- 바탕화면이나 Downloads 폴더는 피하는 것이 좋습니다 (경로에 한글/공백이 있으면 오류 가능)
- 추천 위치: `~/projects/` (Mac) 또는 `C:\projects\` (Windows)

터미널에서 실행:

```bash
# Mac
mkdir -p ~/projects
cd ~/projects

# Windows (CMD)
mkdir C:\projects
cd C:\projects
```

1-2. Next.js 프로젝트 생성

```bash
npx create-next-app@latest cadam
```

이 명령을 실행하면 질문이 나옵니다. 아래와 같이 답하세요:

```
✓ Would you like to use TypeScript?              → Yes
✓ Would you like to use ESLint?                  → Yes
✓ Would you like to use Tailwind CSS?            → Yes
✓ Would you like your code inside a `src/` directory? → Yes
✓ Would you like to use App Router? (recommended)  → Yes
✓ Would you like to use Turbopack for next dev?    → Yes
✓ Would you like to customize the import alias?    → No (기본값 @/ 사용)
```

> **[사실]** PRD 기술 스택: "Next.js 15 (App Router) + TypeScript + Tailwind CSS" cursor_rules: "src/app/ 하위 Next.js App Router 규칙", "Tailwind CSS (인라인 style 금지)" 따라서 위 선택지는 PRD 요구사항을 그대로 반영한 것입니다.

완료 후 확인:

```bash
cd cadam
npm run dev
```

브라우저에서 `http://localhost:3000` 접속 → Next.js 기본 화면이 보이면 성공! `Ctrl + C`로 서버 중지.

🔵 Phase 2: Cursor AI에서 프로젝트 열기 & 환경 설정

2-1. Cursor로 프로젝트 열기

1. Cursor 실행
2. 상단 메뉴 → `File` → `Open Folder`
3. 방금 만든 `cadam` 폴더 선택
4. 왼쪽 사이드바에 프로젝트 파일 트리가 보이면 성공

2-2. Cursor AI 설정 최적화

Cursor Settings 열기:

- Mac: `Cmd + Shift + P` → "Cursor Settings" 검색 → 클릭
- Windows: `Ctrl + Shift + P` → "Cursor Settings" 검색 → 클릭

권장 설정:

- **Models:** `claude-sonnet-4-20250514` 또는 최신 Claude 모델 선택
- **Features → Composer:** 활성화 (멀티파일 동시 편집에 필수)
- **Features → Agent:** 활성화 (자동으로 터미널 명령 실행 가능)

> **[추론]** PRD가 매우 상세하므로 Claude 계열 모델이 긴 문서 이해에 유리합니다. GPT-4o도 사용 가능하지만, 한국어 PRD 처리는 Claude가 더 정확한 경향이 있습니다.

2-3. Cursor Rules 파일 배치 (가장 중요!)

이것은 무엇인가? Cursor AI에게 "이 프로젝트에서는 이런 규칙을 지켜라"고 알려주는 설정 파일입니다. 사람으로 치면 "프로젝트 온보딩 문서"와 같습니다.

Cursor의 터미널 열기: 하단 패널에서 `Terminal` 탭 클릭 (없으면 `Ctrl + ` `)

방법 1 — 터미널 명령어로 생성:

```bash
# 프로젝트 루트에서 실행

# 1) .cursorrules 파일 생성 (레거시 호환)
# → dot_cursorrules.txt 내용을 복사 붙여넣기

# 2) .cursor/rules 파일 생성 (최신 방식)
mkdir -p .cursor
# → cursor_rules.md 내용을 복사 붙여넣기

# 3) docs 폴더 생성 + PRD/와이어프레임 배치
mkdir -p docs
# → PRD_카담_v3_4.md를 docs/PRD.md로 복사
# → wireframe_카담_v3_4.html을 docs/wireframe.html로 복사

# 4) scripts 폴더 생성
mkdir -p scripts
```

방법 2 — Cursor 파일 탐색기에서 직접 생성:

1. 왼쪽 파일 트리에서 프로젝트 루트 우클릭 → `New File`
2. 파일명: `.cursorrules` → 업로드한 dot_cursorrules.txt 내용 붙여넣기
3. `.cursor` 폴더 만들기 → 그 안에 `rules` 파일 생성 → cursor_rules.md 내용 붙여넣기
4. `docs` 폴더 만들기 → `PRD.md`, `wireframe.html` 파일 생성

결과 확인 — 프로젝트 루트 구조:

```
cadam/
├── .cursor/
│   └── rules            ← Cursor AI 규칙 (최신)
├── .cursorrules         ← Cursor AI 규칙 (레거시)
├── docs/
│   ├── PRD.md           ← 기획서 전문
│   └── wireframe.html   ← UI 와이어프레임
├── scripts/             ← (아직 비어있음)
├── src/
│   └── app/             ← Next.js가 만든 기본 구조
├── package.json
├── tailwind.config.ts
├── tsconfig.json
└── ... (기타 설정 파일들)
```

> **[사실]** PRD 13장 File Structure에서 이 구조가 정확히 명시되어 있습니다. cursor_rules에서도 "PRD: @docs/PRD.md", "와이어프레임: @docs/wireframe.html"로 참조 경로가 지정되어 있습니다. 이 파일들이 없으면 Cursor AI가 프로젝트 맥락을 이해할 수 없습니다.

🔵 Phase 3: 패키지(라이브러리) 설치

이것은 무엇인가? 다른 개발자들이 미리 만들어둔 코드 묶음을 가져다 쓰는 것입니다. 요리에 비유하면 "양념장을 직접 만들지 않고 마트에서 사오는 것"과 같습니다.

Cursor 터미널에서 한 번에 실행:

```bash
```

```
# 핵심 UI/상태관리
npm install zustand framer-motion

# Supabase (데이터베이스 + 인증)
npm install @supabase/supabase-js @supabase/ssr

# Rate Limiting (과도한 요청 차단)
npm install @upstash/ratelimit @upstash/redis

# 이메일 발송
npm install resend

# 입력값 검증
npm install zod

# XSS 방지 (Admin 페이지 보안)
npm install dompurify
npm install -D @types/dompurify
```

> **[사실]** 각 패키지가 필요한 이유:
> - zustand: PRD "State Management: Zustand + persist middleware"
> - framer-motion: PRD "전환: Framer Motion slide 좌↔우"
> - supabase: PRD "Supabase (PostgreSQL + Auth)"
> - upstash: PRD "Rate Limiting: Upstash Redis"
> - resend: PRD "이메일 알림 (Resend)"
> - zod: PRD "입력 검증: 서버사이드 zod 스키마"
> - dompurify: PRD "XSS 방지: DOMPurify (Admin 입력)"

설치 확인:

```bash
cat package.json
```

`dependencies` 안에 위 패키지들이 나열되어 있으면 성공.

🔵 Phase 4: Tailwind CSS 커스텀 색상 설정

4-1. tailwind.config.ts 수정

Cursor에서 `tailwind.config.ts` 파일을 엽니다 (왼쪽 파일 트리에서 클릭).

Cursor AI에게 지시하는 방법: Cmd+I (Mac) 또는 Ctrl+I (Windows)를 눌러 인라인 편집 모드를 열고 다음과 같이 입력:

```
이 tailwind.config.ts에 다음 커스텀 색상을 추가해줘:
- primary: #1B3A5C
- accent: #2E86C1
- success: #27AE60
- warning: #F39C12
- danger: #E74C3C
- kakao: #FEE500
```

결과 예시 (AI가 생성할 코드):

```typescript
// tailwind.config.ts
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./src/pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/components/**/*.{js,ts,jsx,tsx,mdx}",
    "./src/app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: '#1B3A5C',
        accent: '#2E86C1',
        success: '#27AE60',
        warning: '#F39C12',
        danger: '#E74C3C',
        kakao: '#FEE500',
      },
      fontFamily: {
        pretendard: ['Pretendard', '-apple-system', 'sans-serif'],
      },
    },
  },
  plugins: [],
};
export default config;
```

> **[사실]** cursor_rules: "색상은 Design System 변수 사용: primary(#1B3A5C)..." PRD 1장: "Colors: Primary #1B3A5C, Accent #2E86C1..." 이 색상을 Tailwind에 등록해야 bg-primary, text-accent 같은 클래스를 쓸 수 있습니다.

🔵 Phase 5: 외부 서비스 가입 & 환경변수 설정

5-1. Supabase 프로젝트 생성

이것은 무엇인가? 데이터베이스 + 사용자 인증을 제공하는 클라우드 서비스입니다. 고객 상담 신청 데이터, 가격 데이터, 관리자 로그인이 모두 여기에 저장됩니다.

단계:

1. https://supabase.com 접속 → Sign Up (GitHub 계정으로 가능)
2. "New Project" 클릭
3. 설정:
   - **Name:** cadam
   - **Database Password:** 강력한 비밀번호 입력 (메모해둘 것!)
   - **Region:** Northeast Asia (Tokyo) — 한국에서 가장 빠름
4. 프로젝트 생성 완료 후 → Settings → API 탭에서:
   - `Project URL` → 이것이 `NEXT_PUBLIC_SUPABASE_URL`
   - `anon public` 키 → 이것이 `NEXT_PUBLIC_SUPABASE_ANON_KEY`
   - `service_role` 키 → 이것이 `SUPABASE_SERVICE_ROLE_KEY` (절대 공개 금지!)

5-2. Upstash Redis 생성

이것은 무엇인가? 같은 사용자가 연속으로 폼을 제출하는 것을 막는 Rate Limiting용 서비스입니다.

단계:

1. https://upstash.com 접속 → Sign Up
2. "Create Database" → Redis 선택
3. 설정:
   - **Name:** cadam-rate-limit
   - **Region:** ap-northeast-1 (Tokyo)
4. 생성 후 REST API 탭에서:
   - `UPSTASH_REDIS_REST_URL`
   - `UPSTASH_REDIS_REST_TOKEN`

5-3. Resend 가입 (이메일 발송)

단계:

1. https://resend.com 접속 → Sign Up
2. API Keys → "Create API Key"

3. 발급된 키 → `RESEND_API_KEY`

> **[가정]** Resend 무료 티어(월 3,000건)로 MVP 기간에는 충분합니다. 도메인 인증 전에는 Resend 제공 이메일(@resend.dev)로 발송됩니다.

5-4. GA4 (Google Analytics) 설정

단계:

1. https://analytics.google.com 접속
2. "관리" → "만들기" → "속성" → 속성명: "카담"
3. 데이터 스트림 → "웹" → URL 입력 (임시로 localhost:3000)
4. 측정 ID (G-XXXXXXXXXX) → `NEXT_PUBLIC_GA_ID`

5-5. 환경변수 파일 생성 (.env.local)

이것은 무엇인가? 비밀번호, API 키 같은 민감 정보를 코드에 직접 쓰지 않고 별도 파일에 보관하는 것입니다. 이 파일은 Git에 올라가지 않아 보안이 유지됩니다.

Cursor에서 프로젝트 루트에 `.env.local` 파일 생성:

```env
```

```
# ================================
# Supabase
# ================================
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJxxxxxxxxxxxx
SUPABASE_SERVICE_ROLE_KEY=eyJxxxxxxxxxxxx

# ================================
# Upstash Redis
# ================================
UPSTASH_REDIS_REST_URL=https://xxxxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AXxxxxxxxxxxx

# ================================
# Resend (이메일)
# ================================
RESEND_API_KEY=re_xxxxxxxxxxxx
ADMIN_EMAIL=iori21y@gmail.com

# ================================
# GA4
# ================================
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX

# ================================
# ISR Revalidation Secret
# ================================
REVALIDATION_SECRET=여기에_긴_랜덤_문자열_입력

# ================================
# 카카오톡 채널 (개설 후 입력, 지금은 placeholder)
# ================================
NEXT_PUBLIC_KAKAO_CHANNEL_URL=https://pf.kakao.com/_placeholder

# ================================
# 운영 전화번호 (확정 후 입력, 지금은 placeholder)
# ================================
NEXT_PUBLIC_PHONE_NUMBER=02-0000-0000
```

REVALIDATION_SECRET 생성 방법:

```bash
```

```
# 터미널에서 실행 (Mac/Linux)
openssl rand -hex 32

# 결과 예: a1b2c3d4e5f6...  이 값을 .env.local에 붙여넣기
```

> **[사실]** PRD 14장에 환경변수 목록이 정확히 명시되어 있습니다. `NEXT_PUBLIC_` 접두사가 붙은 변수만 브라우저에서 접근 가능하고, 나머지는 서버에서만 접근 가능합니다 (보안 설계).

중요 확인: `.gitignore` 파일에 `.env.local`이 포함되어 있는지 확인! (Next.js가 자동으로 추가하지만, 반드시 확인하세요)

🔵 Phase 6: 폴더 구조 생성

PRD에 명시된 폴더 구조를 미리 만들어두면 Cursor AI가 파일을 올바른 위치에 생성합니다.

Cursor 터미널에서 실행:

```bash
# src 하위 폴더 구조 생성
mkdir -p src/components/steps
mkdir -p src/components/cars
mkdir -p src/components/admin
mkdir -p src/constants
mkdir -p src/store
mkdir -p src/lib
mkdir -p src/hooks

# 페이지 폴더 구조 생성
mkdir -p src/app/info
mkdir -p src/app/result
mkdir -p src/app/cars
mkdir -p src/app/privacy
mkdir -p src/app/admin/login
mkdir -p src/app/admin/consultations
mkdir -p src/app/admin/promotions
mkdir -p src/app/admin/prices
mkdir -p src/app/api/consultation
mkdir -p src/app/api/price-range
mkdir -p src/app/api/promotions
mkdir -p src/app/api/revalidate

# 차량 이미지 폴더
mkdir -p public/cars
```

> **[사실]** PRD 13장 File Structure를 그대로 반영한 것입니다. cursor_rules: "본 PRD에 명시되지 않은 폴더 구조나 파일을 임의로 생성하지 말 것" 따라서 이 구조를 정확히 지켜야 합니다.

생성 후 프로젝트 구조 확인 (Cursor 왼쪽 파일 트리):

```
cadam/
├── .cursor/rules
├── .cursorrules
├── docs/
│   ├── PRD.md
│   └── wireframe.html
├── scripts/
├── public/
│   └── cars/           ← 차량 이미지 (나중에 추가)
├── src/
│   ├── app/
│   │   ├── layout.tsx    ← (Next.js 기본 생성)
│   │   ├── page.tsx      ← (Next.js 기본 생성)
│   │   ├── info/
│   │   ├── result/
│   │   ├── cars/
│   │   ├── privacy/
│   │   ├── admin/
│   │   │   ├── login/
│   │   │   ├── consultations/
│   │   │   ├── promotions/
│   │   │   └── prices/
│   │   └── api/
│   │       ├── consultation/
│   │       ├── price-range/
│   │       ├── promotions/
│   │       └── revalidate/
│   ├── components/
│   │   ├── steps/
│   │   ├── cars/
│   │   └── admin/
│   ├── constants/
│   ├── store/
│   ├── lib/
│   └── hooks/
├── .env.local
├── package.json
├── tailwind.config.ts
└── tsconfig.json
```

🔵 Phase 7: Cursor AI로 실제 코딩 시작하기

7-1. Cursor AI 사용법 — 세 가지 모드

모드	단축키	용도	설명
Inline Edit	Cmd+I / Ctrl+I	현재 열린 파일 수정	특정 파일 하나를 고칠 때
Chat	Cmd+L / Ctrl+L	질문 & 코드 생성	설명을 듣거나 코드 조각을 받을 때
Composer (Agent)	Cmd+Shift+I / Ctrl+Shift+I	**멀티파일 자동 생성**	여러 파일을 한 번에 만들 때 (핵심!)

> **[추론]** 카담 프로젝트처럼 여러 파일이 연결된 작업에서는 **Composer(Agent) 모드**가 가장 효율적입니다. 한 번의 지시로 컴포넌트 + 스토어 + 유틸리티를 동시에 생성할 수 있습니다.

7-2. @ 참조 기능 (컨텍스트 제공)

Cursor AI에게 참고할 파일을 알려주는 핵심 기능입니다.

```
@docs/PRD.md          ← PRD 전체를 AI에게 보여줌
@docs/wireframe.html  ← 와이어프레임을 AI에게 보여줌
@src/constants/vehicles.ts  ← 차량 데이터를 AI에게 보여줌
```

사용 예시 (Composer에서):

```
@docs/PRD.md 의 4장을 참고해서
src/constants/vehicles.ts 파일을 만들어줘.
Vehicle 인터페이스, Brand/Category 타입, VEHICLE_LIST 45종 전체,
유틸리티 함수 4개를 포함해야 해.
```

7-3. 첫 번째 작업 — 차량 데이터 상수 파일 생성

왜 이것부터 하는가?

- 이 파일은 다른 파일에 의존하지 않음 (독립적)
- 거의 모든 페이지에서 이 데이터를 참조함
- AI가 정확하게 생성했는지 검증하기 쉬움 (45종 카운트만 하면 됨)

Composer(Agent) 모드 열기: Cmd+Shift+I

프롬프트:

```
@docs/PRD.md 의 "4. Vehicle Data" 섹션을 참고해서
src/constants/vehicles.ts 파일을 생성해줘.

포함해야 할 것:
1. Category, Brand 타입 (export type)
2. Vehicle 인터페이스 (export interface)
3. VEHICLE_LIST 상수 — 현대 19종, 기아 19종, 제네시스 7종 = 총 45종
4. 유틸리티 함수: getVehicleBySlug, getVehiclesByBrand, getVehiclesByCategory, getAllSlugs

PRD에 있는 데이터를 그대로 사용할 것.
any 타입 금지, interface 사용.
```

AI가 코드를 생성하면:

1. "Accept" 버튼을 클릭 (코드가 파일에 반영됨)
2. 생성된 파일을 열어서 차량이 45종인지 확인

7-4. 두 번째 작업 — Zustand 스토어 생성

프롬프트:

```
@docs/PRD.md 의 "5.1. State Management"와 "5.2. Step Flow 상세"를 참고해서
src/store/quoteStore.ts 파일을 생성해줘.

Zustand + persist 미들웨어를 사용하고,
다음 상태를 포함해야 해:
- currentStep (number)
- selectionPath ('car' | 'budget' | null)
- carBrand, carModel, trim
- contractMonths, annualKm
- deposit, prepaymentPct
- monthlyBudget
- name, phone, privacyAgreed
- 각 상태를 업데이트하는 setter 함수들
- resetAll 함수 (스토어 초기화)

localStorage 직접 접근 금지, persist를 통해서만.
any 타입 금지.
```

7-5. 세 번째 작업 — 유틸리티 라이브러리

프롬프트:

```
@docs/PRD.md 를 참고해서 다음 파일들을 한 번에 생성해줘:

1. src/lib/supabase.ts — Supabase 클라이언트 (환경변수 사용)
2. src/lib/leadScore.ts — PRD 3.4장의 리드 점수 계산 로직 그대로 구현
   (서버사이드 전용 주석 포함)
3. src/lib/rateLimit.ts — PRD 8.2장의 Upstash Redis Rate Limiter 그대로 구현
4. src/lib/utils.ts — 전화번호 하이픈 제거, 포맷팅, IP SHA-256 해시 함수
5. src/lib/gtag.ts — GA4 이벤트 트래킹 유틸리티
6. src/lib/notification.ts — Resend 이메일 발송 (PRD 8.5장)
7. src/lib/kakao.ts — Phase 2 placeholder (함수 시그니처 + 주석만)

모든 파일은 TypeScript, any 타입 금지.
kakao_sent 관련 로직 작성 금지.
```

7-6. 네 번째 작업 — 훅(Hooks) 생성

프롬프트:

```
@docs/PRD.md 를 참고해서:

1. src/hooks/useHydrated.ts — SSR Hydration mismatch 방지 훅
   (Zustand persist가 클라이언트에서만 작동하므로 필요)
2. src/hooks/useLeaveIntent.ts — 이탈 감지 훅
   Desktop: 마우스 Y ≤ 0
   Mobile: popstate 이벤트
   조건: Step 2 이상, 세션당 1회만 트리거
```

🔵 Phase 8: 데이터베이스 설정 (Supabase)

8-1. 테이블 생성

Supabase Dashboard(https://supabase.com/dashboard)에 로그인 후:

1. 왼쪽 메뉴 → **SQL Editor** 클릭
2. "New Query" 클릭
3. 아래 SQL을 Cursor AI에게 생성시키거나, 직접 붙여넣기

Cursor Composer에서 SQL 생성 요청:

```
@docs/PRD.md 의 "3. Database & Security" 섹션을 참고해서
Supabase에서 실행할 SQL 스크립트를 생성해줘.

포함할 것:
1. consultations 테이블 (22개 컬럼 전부)
2. price_ranges 테이블 (conditions jsonb 포함)
3. promotions 테이블
4. notification_log 테이블
5. 3.2장의 RLS 정책 전부
6. 3.3장의 인덱스 7개 전부

파일 위치: scripts/seed.ts가 아니라 별도 SQL 파일로.
```

생성된 SQL을 Supabase SQL Editor에 붙여넣고 "Run" 클릭.

8-2. 관리자 계정 생성

Supabase Dashboard → **Authentication** → **Users** 탭:

1. "Add User" → "Create New User"
2. Email: `iori21y@gmail.com`
3. Password: 원하는 강력한 비밀번호 입력
4. "Auto Confirm User" 체크
5. "Create User" 클릭

🔵 Phase 9: 첫 화면 만들기 (Step 플로우 시작)

여기서부터 본격적인 UI 개발입니다.

9-1. 루트 레이아웃 수정

프롬프트:

```
@docs/PRD.md 1장과 @docs/wireframe.html 을 참고해서
src/app/layout.tsx를 수정해줘.

- Pretendard 웹폰트 (CDN, font-display: swap, preload)
- GA4 스크립트 (next/script, NEXT_PUBLIC_GA_ID)
- 메타데이터: title "카담(CADAM) | 장기렌터카 맞춤 견적"
- body에 max-width 1024px 중앙 정렬 컨테이너
- 모바일 퍼스트: min-width 360px

Tailwind CSS만 사용, 인라인 style 금지.
```

9-2. Step 공통 레이아웃 + ProgressBar

프롬프트:

```
@docs/PRD.md 5.2장 "공통 레이아웃"과 @docs/wireframe.html 을 참고해서:

1. src/components/steps/StepLayout.tsx
   - 100vh 풀스크린, 배경 #FFFFFF
   - 상단: ProgressBar
   - 중앙: children (질문 + 선택지)
   - 하단: 이전/다음 버튼 (고정)
   - Framer Motion slide 좌↔우 전환
   - 'use client' 필수

2. src/components/steps/ProgressBar.tsx
   - currentStep / totalSteps (6)
   - accent 색상 (#2E86C1) 프로그레스 바

와이어프레임의 레이아웃과 간격을 최대한 따르되,
스타일은 반드시 Tailwind CSS 클래스로 구현.
인라인 style이나 CSS 변수 사용 금지.
```

9-3. Step 1 ~ Step 6 컴포넌트

각 Step을 하나씩 만드는 것이 안전합니다. 한 번에 전부 요청하면 누락이 생깁니다.

Step 1 프롬프트 예시:

```
@docs/PRD.md 5.2장 "Step 1: 선택 분기"와
@docs/wireframe.html 의 Step 1 화면을 참고해서
src/components/steps/Step1Branch.tsx를 만들어줘.

- "당신에게 더 중요한 선택은 무엇인가요?"
- 🚙 "차종 먼저 선택할게요" → selectionPath: 'car'
- 💰 "월 예산 먼저 정할게요" → selectionPath: 'budget'
- Zustand 스토어(@src/store/quoteStore.ts)와 연결
- 선택 시 다음 Step으로 이동
- 와이어프레임의 카드 스타일 참고, Tailwind CSS로 구현

'use client' 필수, any 금지.
```

나머지 Step도 같은 패턴으로:

- Step 2A → Step2Car.tsx (브랜드탭 + 카테고리필터 + 차종그리드 + 트림시트)
- Step 2B → Step2Budget.tsx (예산 5단계 카드)
- Step 3 → Step3Period.tsx (36/48/60개월)

- Step 4 → Step4Mileage.tsx (1만/2만/3만/4만km)
- Step 5 → Step5Payment.tsx (보증금 + 선납금)
- Step 6 → Step6Contact.tsx (이름+연락처+동의+제출)

9-4. 메인 페이지 조립

프롬프트:

```
src/app/page.tsx를 수정해서
Step 1~6을 StepLayout 안에서 currentStep에 따라 조건부 렌더링해줘.

@src/store/quoteStore.ts의 currentStep 사용.
@src/components/steps/ 하위 컴포넌트들을 import.
@src/hooks/useHydrated.ts로 Hydration 처리.

selectionPath에 따라 Step 2A(차종) 또는 Step 2B(예산) 분기.
'use client' 필수.
```

🔵 Phase 10: 이후 작업 순서 요약

Phase 9까지 완료하면 메인 견적 플로우가 화면에 보입니다.
이후 작업은 이 순서를 권장합니다:

```
10. API Routes 구현 (/api/consultation, /api/price-range 등)
11. 결과 페이지 (src/app/result/page.tsx)
12. Toast 컴포넌트 + 이탈 방지 모달
13. 정보/프로모션 페이지 (/info)
14. SEO 차종별 상세 페이지 (/cars/[slug])
15. 개인정보 처리방침 (/privacy)
16. Middleware (쿠키 수집)
17. Admin 로그인 + 대시보드
18. Admin 상담 관리 (CRM)
19. Admin 프로모션/가격표 관리
20. GA4 이벤트 연동
21. 테스트 + 배포 (Vercel)
```

⚠️ Cursor AI 사용 시 핵심 팁

팁 1: 작은 단위로 지시하라

```
❌ "PRD 전체를 보고 프로젝트를 완성해줘"
```

✅ "PRD 5.2장의 Step 3을 만들어줘. @docs/wireframe.html 참고."

팁 2: @ 참조를 항상 사용하라

❌ "차량 데이터를 사용해서..."
✅ "@src/constants/vehicles.ts 의 VEHICLE_LIST를 사용해서..."

팁 3: 규칙을 명시적으로 상기시켜라

"Tailwind CSS만 사용, 인라인 style 금지.
any 타입 금지, interface 사용.
'use client' 필수."

팁 4: 에러가 나면 에러 메시지를 그대로 복사해서 물어봐라

"터미널에서 이런 에러가 나왔어: [에러 메시지 붙여넣기]
이거 어떻게 해결해?"

팁 5: AI가 만든 코드를 바로 수락하지 말고 검토하라

특히 확인할 것:

- `any` 타입이 쓰이지 않았는지
- `style={{}}` 인라인 스타일이 없는지
- `import` 경로가 올바른지 (`@/` 접두사)
- 차량 데이터를 하드코딩하지 않고 `VEHICLE_LIST`를 참조하는지

팁 6: 자주 저장하고, 자주 확인하라

```bash
# 변경사항 저장 후 개발 서버 실행
npm run dev
# http://localhost:3000 에서 화면 확인
```

팁 7: Git으로 중간 저장하라

```bash
# 큰 기능이 완성될 때마다
git add .
git commit -m "Step 1-3 컴포넌트 완성"
```

문제가 생기면 `git checkout .`으로 마지막 저장 시점으로 되돌릴 수 있습니다.

📋 체크리스트 (진행 확인용)

Phase 0: 사전 준비

- [ ] Node.js LTS 설치 완료
- [ ] Git 설치 완료
- [ ] Cursor AI 설치 + 계정 생성

Phase 1: 프로젝트 생성

- [ ] `npx create-next-app@latest cadam` 실행
- [ ] `npm run dev`로 기본 화면 확인

Phase 2: Cursor 환경설정

- [ ] `.cursorrules` 파일 배치
- [ ] `.cursor/rules` 파일 배치
- [ ] `docs/PRD.md` 배치
- [ ] `docs/wireframe.html` 배치

Phase 3: 패키지 설치

- [ ] zustand, framer-motion 설치
- [ ] supabase 관련 패키지 설치
- [ ] upstash, resend, zod, dompurify 설치

Phase 4: Tailwind 설정

- [ ] 커스텀 색상 6개 등록
- [ ] Pretendard 폰트 설정

Phase 5: 외부 서비스

- [ ] Supabase 프로젝트 생성 + 키 발급
- [ ] Upstash Redis 생성 + 키 발급
- [ ] Resend 가입 + API 키 발급
- [ ] GA4 속성 생성 + 측정 ID 발급
- [ ] `.env.local` 파일 작성 완료

Phase 6: 폴더 구조

- [ ] PRD 13장 기준 폴더 전부 생성

Phase 7: 핵심 코드 생성

- [ ] vehicles.ts (45종 확인)

- [ ] quoteStore.ts
- [ ] lib/ 유틸리티 7개 파일
- [ ] hooks/ 2개 파일

Phase 8: 데이터베이스

- [ ] Supabase 테이블 4개 생성
- [ ] RLS 정책 적용
- [ ] 인덱스 7개 생성
- [ ] 관리자 계정 생성

Phase 9: 첫 화면

- [ ] layout.tsx 수정 (폰트, GA4)
- [ ] StepLayout + ProgressBar
- [ ] Step 1 ~ Step 6 컴포넌트
- [ ] page.tsx에서 Step 조립
- [ ] `npm run dev`로 전체 플로우 확인!